Where a Node Sits Should Mean Something

Turning 3D graph layout from a picture into structured metadata both a human and a model can read

Michal Malček, with Claude Opus and a local Gemma-4 12B · July–August 2026

Abstract

3D knowledge-graph visualisations are usually decoration: a layout algorithm produces coordinates, a human looks at them, and the model that shares the graph sees a flat list of entities with no notion of where anything sits. We asked whether the geometry could carry information for both parties at once.

The starting point was a defect, not an idea. Our knowledge graph **imploded**: the more connected an entity was, the further toward the centre it was pulled, so the interesting part of the graph collapsed into an unreadable dense core while disconnected nodes drifted at random. The human could not fly through it; the model could not see it at all.

Three findings came out of fixing that:

- **The implosion was caused by degree-blind repulsion**, and the standard fix — scaling repulsion by node degree — converted the collapse into an orbital structure with legible bands.
- **Position is only useful to a language model if it is quantised into tokens.** Raw floats are not actionable; discrete tier and cluster labels are. This was grounded against the literature and then confirmed by asking the local model directly.
- **The same computation must run on both sides.** The human's visual layout and the model's stored metadata are computed by two separate code paths, and if they drift, the human and the model are looking at different universes while believing they share one.

The result is deployed: every entity in two graphs carries a stored `importance_tier` and `cluster_id`, exposed through every read path, used by the model as a search-space filter and by the human as a flight map.

---

1. The defect that started it

The human's observation, verbatim: *"Our KG doesn't expand into space, it densifies — hard to fly even in flight-mode."*

Reading the layout engine confirmed the cause and showed it was the opposite of what was wanted. Two forces both pulled connected nodes inward:

force	behaviour
shell assignment	`octave = ceil(log2(degree + 1))` — high degree → low octave → **inner shell**
centre gravity	connected nodes received full centre-pull; isolated nodes 10%

Net effect: connectivity was treated as a reason to move toward the centre. The most important entities — the ones with the most relationships — ended up in the smallest volume, occluding each other, while weakly-connected nodes floated without structure.

The human's intended inversion was precise: connectivity should *spread* entities through space, and weakly-linked nodes should be pushed to outer orbitals, so that isolated material sits visibly at the periphery instead of scattering.

2. Grounding before building

The first fix attempted was to add an outward force on top of the existing ones. It was too weak, and grounding explained why: it was fighting the springs rather than correcting the term that was wrong.

The established answer:

> **Standard force-directed layouts implode hubs because edge attraction*
>
> *overpowers node repulsion.**

Four candidate algorithms were compared:

algorithm	orbital shelling	implosion prevention	human-readable	model-friendly
ForceAtlas2 (LinLog)	high, organic	excellent	excellent	excellent
Radial / k-core shells	maximum, strict	absolute	good	good
Hyperbolic (Poincaré)	high, mathematical	absolute	**poor — distorted**	maximum
Sugiyama hierarchical	none — rigid planes	—	—	—

ForceAtlas2 LinLog was chosen for a reason that decided the whole project: it is the only option in the table that scores well for *both* readers. Hyperbolic embedding is mathematically superior for a model and unusable for a human, and a tool that only one of the two parties can read defeats the purpose of a shared graph.

The mechanism is a one-line change in principle: **make Coulomb repulsion scale with degree** (`repulsion ×= 1 + log2(degree)`) rather than adding a separate outward force. Hubs then push each other and their neighbourhoods apart, producing an orbital halo instead of a collapsed core.

Hyperbolic space is parked rather than dismissed: it remains the right answer if a model-only view is ever built.

3. The anchor, and why it is not arbitrary

The human's insight: **pin the highest-degree `person` entity at the origin** on every simulation step in gravity mode.

This sounds like vanity and is not. It makes *distance from centre* a clean, single-axis importance spectrum instead of an accident of initial conditions. Everything else arranges itself gravitationally around a fixed point, so the question "how central is this to the work?" has a visual answer.

The empirical result is the interesting part. That anchored person is **not** hardcoded as important — the anchor is selected by degree. He lands at the centre because a knowledge graph genuinely built around one person's work *has* him as its second-highest-degree node (156 relationships), behind only the project itself. The centrality is emergent, and it would move if the work moved.

Confirmed visually on live data: the force view was *"hard to read, imploded"*; the gravity view was *"clearly readable at a glance — concepts and lessons spread from centre by importance."* The disconnected cluster of abandoned experiments settled cleanly at the periphery, which made it prunable for the first time.

4. The real question: does position mean anything to a model?

A pretty layout helps one reader. The hypothesis worth testing was the human's: *could a mathematically-derived position carry precomputed value the model can consume directly — filter on, or use to discover entities that share a location and therefore share something non-obvious?*

4.1 What the literature says

The idea maps onto established work: hyperbolic graph embeddings, GraphSAGE and node2vec with dimensionality reduction, and GNN→LLM hybrids that feed a computed coordinate vector to a language model as a spatial token.

The practical constraint that shaped our design:

> **A language model will not reliably compute Euclidean distance. Do not give it*
>
> *raw floats — quantise the space into discrete tokens it can pattern-match.**

So: not `[10.23, 14.88, 1.02]` but a **sector and layer tag**. String equality is something a transformer does natively; `sqrt((x₂−x₁)² …)` is not.

4.2 What the model itself said

Rather than assume, we asked the local model directly whether spatial metadata would help it or be noise. Its answer, and it named its own two preferred fields:

> **"Highly beneficial, not noise. Spatial metadata is a global heuristic to*
>
> *prioritise search space and identify structural anomalies without exhaustive*
>
> *traversal."**

- `importance_tier` — a relevance filter: which nodes to expand first.
- `cluster_id` — semantic locality: the node's domain, or a flag that it is a bridge or an outlier.

This converged with the literature independently: bind structural meaning to discrete labels, precompute the geometry, hand the model tokens rather than arithmetic.

Note on the limits of this evidence. A model asked whether a proposed feature would help it is not a controlled measurement, and a separate study of ours found that a model's self-report about its own operating rules was wrong 7 times in 8. The value here is that its answer was *specific and actionable* — it named fields we had not proposed — not that it was enthusiastic. The behavioural test is whether the fields are used in practice, which §7 addresses and which remains the weakest link in this work.

5. What was built

Two stored fields, one standard computation each, on both graphs.

`importance_tier` — degree percentiles rather than absolute thresholds (p95 → CORE, p80 → MAJOR, p50 → MINOR, p20 → PERIPHERAL, else ISOLATED). Percentiles rather than fixed cut-points so the bands stay meaningful as the graph grows.

`cluster_id` — connected components via iterative BFS, with labels remapped by **descending component size**, so `cluster_id = 0` is always the giant component. That stability matters: it makes "is this attached to the main body of knowledge?" a constant question with a constant answer, rather than a label that reshuffles whenever an entity is added.

An explicit non-decision: recursive DFS would have been shorter and would have blown the stack on the giant component. Louvain community detection was considered and deferred — connected components is free, catches the disconnected island, and finer domain clustering can come later if it is ever needed.

5.1 The lockstep constraint — the most important line in the code

The tiers are computed **twice**: once client-side for the 3D view, once server-side for storage. The two implementations must produce identical output, and both files carry a comment saying so.

If they diverge, the human's orbital view and the model's reasoning disagree about which entities are central — and neither party has any way to notice. Everyone keeps working, confidently, on two different maps.

This is the same failure class as every other silent defect we have recorded: no error, no warning, just two components quietly disagreeing. It is the reason the computation is duplicated deliberately and documented in both places rather than being left as an implementation detail.

5.2 Distribution on live data

Knowledge graph, 405 entities:

tier	entities
CORE	22
MAJOR	70
MINOR	115
PERIPHERAL	144
ISOLATED	54

Code graph, 2,985 entities: 156 CORE · 478 MAJOR · 1,244 MINOR · 1,090 PERIPHERAL · 17 ISOLATED.

Cluster distribution, knowledge graph: **344 of 405 in the giant component**, 35 in a second island, and a long tail of components of size 2–4. That 35-node island turned out to be a family of abandoned experiments — visible as a peripheral clump in the 3D view, and queryable as `cluster_id = 1` by the model. Both readers see the same thing by different means, which was the entire point.

6. The failure mode nobody warns you about

Tiers are computed **on demand**, not on write. Any entity added or edited invalidates every percentile in the table, because a percentile is a property of the whole distribution rather than of one row.

Measured after a single session of ordinary work: **1,512 of 3,039 code entities had a null tier.** Half the graph had silently lost its spatial metadata, and nothing reported it.

Worse, a null tier is easy to misread as *"this entity is unimportant"* when it actually means *"this entity has not been measured."* A model filtering on tier would have quietly skipped half the codebase.

Fixed at the source: the indexer now schedules a recompute 30 seconds after a batch of file changes settles. Verified live, 1,513 → 0.

The generalisable finding: a derived global property is not a field, it is a cache — and every cache needs an invalidation story written down before it ships. We shipped without one and the gap was invisible for weeks.

7. What is proven and what is not

Proven.

- The implosion had a specific cause and a standard fix, and the fix works: the graph is flyable and importance is legible at a glance.
- Position can be made machine-readable by quantising it into discrete labels; the fields exist, are stored, are exposed through every read path, and are filterable in one call.
- The two computations stay in lockstep, and the staleness that broke them is now fixed at the source rather than patched.

Not proven, and stated plainly.

- **Whether the model's reasoning is measurably better with the fields than without them.** It says they help; it uses them when asked to; there is no A/B measurement of task outcomes with and without spatial metadata. That is the experiment this work still owes, and our own rules research is the reason we distrust the self-report: a model can use a feature without being able to judge whether it helps.
- **Whether coordinate coincidence discovers anything.** The most interesting claim — that entities landing in the same region share something non-obvious — is untested. The sector/layer quantisation needed for it was deferred.

- **Whether tiers improve retrieval ranking.** They are stored but no ranking function currently reads them. A known limitation of our search is that cross-layer fusion has nothing to fuse and falls back to round-robin ordering; tier is exactly the missing tie-breaker, and wiring it in is unfinished work rather than a result.

8. What would settle it

- **A/B the model on graph-traversal tasks** with tier and cluster present versus absent, measuring path length and answer correctness rather than asking.
- **Quantise coordinates into sector tags** and test whether spatial coincidence surfaces relationships that neither semantic search nor graph traversal finds.
- **Use tier as the tie-breaker** in cross-layer result fusion, and measure whether ranking improves on queries with known answers.

Each is a measurement rather than a build, which is the right order — and the reason none of them is presented here as a result.

Appendix: how this was run

Over roughly two weeks in July 2026, alongside other work. The layout change was opt-in from the start (a third mode beside the existing two) so the working view was never disturbed and the comparison could be made live, side by side. The agreement was explicit: verify for real, then either integrate cleanly or remove the experiment entirely — no half-tuned constants left behind.

Grounding came from a search-grounded model and was checked against the primary sources it cited. The local model's assessment was obtained by asking it directly, in its own working context, with the answer recorded verbatim including the parts that did not support the hypothesis.

Every number in §5.2 and §6 was queried from the live database while writing this document, not recalled.